

Modeling Tabular Data

Data and instructions

Use these two public datasets.

- **UCI Wine**, DOI [10.24432/C5PC7J](https://doi.org/10.24432/C5PC7J), loaded from `data/tabular/wine/wine.data`: 178 observations, 13 continuous chemical measurements, three cultivar labels, and no missing values.
- **UCI Dry Bean**, DOI [10.24432/C50S4B](https://doi.org/10.24432/C50S4B), loaded from `data/tabular/dry_bean/Dry_Bean_Dataset.arff`: 13,611 observations, 16 numeric image-derived shape measurements, seven bean-variety labels, and no missing values.

Full metadata, attribution, limitations, and loading instructions are in `homework/tabular/README_data.md`.

For both datasets, use a class-stratified 80/20 train/test split. For penalty selection in regularized models, you may choose to split the outer training set further into class-stratified 80/20 inner-training and validation subsets.

Evaluation metrics

Use the following definitions for possible evaluations. Let $y_i \in \{1, \dots, K\}$ be the true class, $\hat{y}_i$ the predicted class, and $\hat{p}_{ik}$ the predicted probability for class k . Define the $K \times K$ confusion matrix by

$$C_{ab} = \sum_{i=1}^n \mathbb{I}\{y_i = a, \hat{y}_i = b\}.$$

Rows are true classes and columns are predicted classes. For class label a , let $n_a = \sum_b C_{ab}$ and $\hat{n}_a = \sum_b C_{ba}$. Then

$$\text{accuracy} = \frac{\sum_a C_{aa}}{n}, \quad R_a = \text{recall}_a = \frac{C_{aa}}{n_a}, \quad \text{balanced accuracy} = \frac{1}{K} \sum_{a=1}^K R_a.$$

For each class, define

$$P_a = \frac{C_{aa}}{\hat{n}_a}, \quad F1_a = \frac{2P_a R_a}{P_a + R_a}, \quad \text{macro F1} = \frac{1}{K} \sum_{a=1}^K F1_a.$$

Use $P_a = 0$ when no observation is predicted as class a , and use $F1_a = 0$ whenever $P_a + R_a = 0$. Finally, the multiclass log loss is

$$\log \text{loss} = -\frac{1}{n} \sum_{i=1}^n \log \hat{p}_{i, y_i}.$$

Accuracy weights every observation equally. Balanced accuracy weights every class recall equally, while macro F1 also accounts for class-specific precision. Report per-class recall as the vector $(R_1, \dots, R_K)$.

Submit one PDF containing derivations, tables, required figures, and interpretation, together with readable Python code.

Problem 1: Multinomial logistic regression

For $K \geq 3$ classes and label y_i , we use the one-hot code $Y_{ik} = \mathbb{I}\{y_i = k\}, k = 1, \dots, K$ for softmax regression. Define $z_{ik} = x_i^\top \beta_k + b_k$ and the corresponding softmax probabilities. Starting from the categorical likelihood:

- (a) derive the multiclass cross-entropy in a numerically stable log-sum-exp form;
- (b) derive the gradients with respect to every β_k and b_k ;
- (c) derive the L2 gradient contribution and the L1 proximal soft-thresholding update when intercepts are not penalized.

Problem 2: Scratch logistic implementation and model selection

Implement a reusable `ScratchMultinomialLogisticRegression` with explicit softmax, cross-entropy, gradient updates, and either L1 proximal or L2 gradient regularization. Fit unregularized, L1, and L2 models to both Wine and Dry Bean datasets. Fit the feature standardization only on the relevant training partition, then apply that fitted transformation to its validation or test partition.

For each dataset and each penalty, evaluate the same grid of at least eight logarithmically spaced positive values of λ using only the inner-training/validation split. Select the smallest λ attaining the minimum validation multiclass log loss. If that minimum is at a grid boundary, extend the grid or report explicitly that the current grid does not bracket an interior optimum. Produce, for each dataset:

- (a) training and validation data-loss trajectories for the unregularized fit and the selected L1 and L2 fits;
- (b) validation log loss against λ for L1 and L2;
- (c) the number of nonzero coefficients against λ ;
- (d) Do NOT standardize features for the test data when you do inference on the trained models (i.e., use fitted standardization transform on the training data). Why?
- (e) a held-out comparison of the refitted unregularized and selected regularized models.

Problem 3: Head-to-head library verification

Fit matching scikit-learn multinomial logistic models on each outer training set. For the scratch objective

$$\frac{1}{n} \sum_{i=1}^n \ell_i(B, b) + \lambda \|B\|_1 \quad \text{or} \quad \frac{1}{n} \sum_{i=1}^n \ell_i(B, b) + \frac{\lambda}{2} \|B\|_F^2,$$

derive the matching relationship among scikit-learn's C , n , and λ . Compare scratch and library accuracy, balanced accuracy, macro F1, multiclass log loss, per-class recall, and confusion matrices. After applying the same centering convention, compare coefficient correlation and relative Frobenius error; for L1 also compare the selected supports. Explain any gap when possible.

Problem 4: SAMME AdaBoost

SAMME [1] stands for *Stagewise Additive Modeling using a Multi-class Exponential loss*. It is a direct, discrete multiclass extension of AdaBoost. Code a depth-one decision tree prediction $h_m(x) = k$ as the vector $g_m(x)$ with coordinate k equal to 1 and every other coordinate equal to $-1/(K-1)$.

Different from the regression based on softmax, SAMME uses the distinct sum-to-zero code

$$\tilde{Y}_{ik} = \begin{cases} 1, & y_i = k, \\ -1/(K-1), & y_i \neq k, \end{cases} \quad \sum_k \tilde{Y}_{ik} = 0,$$

and its empirical multiclass exponential loss is

$$\mathcal{L}_{\text{exp}}(F) = \frac{1}{n} \sum_{i=1}^n \exp \left\{ -\frac{1}{K} \tilde{Y}_i^\top F(x_i) \right\}.$$

Similar to AdaBoost, this is a classification surrogate and is *not* multiclass log loss. Starting from $F^{(0)} = 0$ and normalized weights $w_i^{(1)} = 1/n$, round m of SAMME does the followings:

- (i) fits a K -class depth-one decision tree h_m using $w_i^{(m)}$;
- (ii) computes

$$\epsilon_m = \sum_{i=1}^n w_i^{(m)} \mathbb{I}\{y_i \neq h_m(x_i)\};$$

- (iii) for learning rate $0 < \nu \leq 1$, sets

$$\alpha_m = \nu \left[\log \frac{1 - \epsilon_m}{\epsilon_m} + \log(K-1) \right];$$

- (iv) updates and renormalizes

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp\{\alpha_m \mathbb{I}(y_i \neq h_m(x_i))\}}{\sum_j w_j^{(m)} \exp\{\alpha_m \mathbb{I}(y_j \neq h_m(x_j))\}}.$$

After M rounds,

$$V_k(x) = \sum_{m=1}^M \alpha_m \mathbb{I}\{h_m(x) = k\}, \quad \hat{y}(x) = \arg \max_k V_k(x).$$

Derive the coefficient from the multiclass exponential-loss stage and show that the one-dimensional stage objective and its minimizer are

$$(1 - \epsilon_m) e^{-\beta_m/(K-1)} + \epsilon_m e^{\beta_m/(K-1)^2}, \quad \beta_m = \frac{(K-1)^2}{K} \left[\log \frac{1 - \epsilon_m}{\epsilon_m} + \log(K-1) \right].$$

Show why converting from sum-to-zero vector coding to the equivalent class-vote coding removes the prefactor, and why shrinkage gives the algorithmic α_m above. Show that it is positive exactly when $\epsilon_m < 1 - 1/K$. Explain the better-than-chance condition, the relative increase in weight after an error, the $K = 2$ case, and the handling of zero error or an invalid weak learner.

For probability diagnostics, use the library-compatible normalized coded score

$$D(x) = \frac{\sum_m \alpha_m g_m(x)}{\sum_m \alpha_m}, \quad \hat{p}_k(x) = \text{softmax}_k \left(\frac{D(x)}{K-1} \right).$$

Make clear that log loss computed from $\hat{p}$ is a calibration diagnostic, whereas $\mathcal{L}_{\text{exp}}$ is the stagewise training objective.

On Dry Bean, fit scratch and scikit-learn SAMME models with 80 depth-one decision trees, $\nu = 0.5$. Use the inner split to plot staged training and validation $\mathcal{L}_{\text{exp}}$; staged training and validation multiclass log loss from the score-to-probability mapping; and the weighted tree error with the $1 - 1/K$ threshold. Refit on the outer training set and report test accuracy, with macro F1 and log loss as secondary diagnostics. Explain agreement or disagreement.

Problem 5: Multiclass XGBoost-style Newton boosting

This problem will not be graded. You do not have to submit your solution for Problem 5. Have fun.

For the softmax objective, derive $g_{ik} = p_{ik} - \mathbb{I}(y_i = k)$ and the exact per-observation Hessian $\text{diag}(p_i) - p_i p_i^\top$. For the scratch implementation, use the diagonal approximation $h_{ik} = p_{ik}(1 - p_{ik})$. Derive the resulting L2-regularized constant leaf value and split gain, defining G , H , λ , and γ .

On Dry Bean, compare a scratch exact-threshold, depth-wise implementation with `XGBClassifier` using its `multi:softprob` objective, 40 boosting rounds, learning rate 0.10, depth 2, minimum child Hessian 1, $\lambda = 1$, $\gamma = 0$, row subsampling 0.8, and column subsampling 0.8. Use 10 as the scratch minimum leaf count; use XGBoost's histogram tree method and otherwise document defaults that cannot be matched exactly. In particular, native multiclass XGBoost uses the diagonal Hessian upper bound $2p_{ik}(1 - p_{ik})$ [2], not the scratch approximation above, so equal numerical values of minimum child Hessian and λ do not make those controls exactly equivalent. On the inner split, plot training and validation multiclass log loss against boosting round for both implementations. Refit on the outer training set and report test accuracy, balanced accuracy, macro F1, multiclass log loss, and both confusion matrices. Explain why close results are possible without identical trees or probabilities.

References

- [1] J. Zhu, H. Zou, S. Rosset, and T. Hastie, *Multi-class AdaBoost*, *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009. doi: [10.4310/SII.2009.v2.n3.a8](https://doi.org/10.4310/SII.2009.v2.n3.a8).
- [2] XGBoost developers, *Demo for creating a customized multi-class objective function*, XGBoost 3.4.1 documentation. https://xgboost.readthedocs.io/en/stable/python/examples/custom_softmax.html.